Studio Nx

Backend(FastAPI/Python) 55 — retention.main()

2026-10-12

Backend(FastAPI/Python) 55 — `retention.main()` (+ 처리방침 버전 2026-10-08)

항목	내용
파일	`studionx/backend/app/workers/retention.py` / 처리방침: `privacy.html`, `home.html`, `join.js`, `schemas/auth.py`, `docs/Privacy_Policy_Draft.md`
작업	P1-12 문의 API: 보관 기간 정리를 하루 한 번 실행하는 명령 (처리방침 §4)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	54 `purge_old_inquiries()` · 56 프로젝트 요청 스키마 (P1-13)

1. 하는 일

명령 한 줄로 **보관 기간 정리 두 가지**를 차례로 실행하고, 결과를 한 줄로 출력합니다.

```
cd studionx/backend
..\.venv\Scripts\python -m app.workers.retention
# → withdrawn accounts: 0, inquiries: 0
```

차례	정리	함수
①	탈퇴한 지 3일이 지난 계정의 개인 정보를 비웁니다	49번 `purge_withdrawn_accounts` (P1-09)
②	답변한 지 1년, 또는 답변이 없다면 받은 지 1년이 지난 문의를 지웁니다	54번 `purge_old_inquiries`

정리하다가 오류가 나면 예외를 그대로 올려 보냅니다. 그러면 Sentry에 보고되고, 명령은 실패로 끝납니다. 예약 작업이 실패를 알아챌 수 있게 하기 위해서입니다.

2. 언제 쓰나

운영체제의 예약 작업으로 **하루에 한 번** 실행합니다 (Windows 작업 스케줄러나 리눅스 cron). 예약 설정은 **배포할 때** 정합니다 (템플릿 머리말). 지금은 손으로 한 번 실행해서 동작을 확인했습니다. 실제 DB에서 Sentry를 끄고 실행했더니 `withdrawn accounts: 0, inquiries: 0`이 출력되고 종료 코드 0으로 끝났습니다 (지울 것이 없는 상태).

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import delete, func, or_, select
from sqlalchemy.orm import Session

from app.core.config import get_settings
from app.core.sentry import init_sentry
from app.db.session import SessionLocal
```

```
from app.models import Inquiry, User
from app.models.user import UserStatus
from app.services.auth_service import REJOIN_WAIT_DAYS, release_withdrawn
```

`init_sentry`와 `SessionLocal` 줄을 이번에 추가했습니다.

함수

```
def main() -> None:
    init_sentry(get_settings())
    now = datetime.now(UTC)
    with SessionLocal() as db:
        accounts = purge_withdrawn_accounts(db, now)
        inquiries = purge_old_inquiries(db, now)
    print(f"withdrawn accounts: {accounts}, inquiries: {inquiries}")
```

파일 맨 아래의 `if __name__ == "__main__": main()`은 템플릿 그대로입니다. `python -m app.workers.retention`으로 실행하면 `main()`이 호출됩니다.

문법 풀이

import 문법	뜻
`from app.core.sentry import init_sentry`	서버(P1-01)와 같은 Sentry 설정입니다. 명령으로 따로 실행될 때에도 오류를 보고하기 위해서 씁니다
`from app.db.session import SessionLocal`	DB 세션을 만드는 공장입니다. 서버 요청 밖에서 직접 세션을 만듭니다

식	뜻
`with SessionLocal() as db:`	세션을 열고, 블록이 끝나면(오류가 나더라도) 닫습니다
`datetime.now(UTC)`	시간대가 붙은 지금 시각입니다. 49번과 54번이 이것을 UTC로 바꿔 비교합니다
`f"…{accounts}…"`	문자열 안에 값을 끼워 넣습니다
`python -m app.workers.retention`	파일을 **모듈**로 실행합니다. `app` 패키지를 찾을 수 있도록 `studionx/backend` 폴더에서 실행합니다

4. 흐름

1. Sentry를 켭니다.
2. 지금 시각을 한 번 잽니다.
3. 세션을 열고 탈퇴 계정 정리와 문의 정리를 차례로 실행합니다 (각각 저장합니다).
4. 결과를 한 줄로 출력합니다. 오류가 나면 그대로 올려 보냅니다.

5. 왜 이렇게

선택	이유
기준 시각을 한 번만 잼	두 정리가 같은 "지금"을 기준으로 삼게 해서, 기록을 읽을 때 헷갈리지 않게 합니다
오류를 삼키지 않음	정리가 조용히 실패하면 처리방침에서 약속한 삭제를 지키지 못하게 됩니다. Sentry 보고와 실패 종료 코드로 알아챌 수 있게 했습니다

선택	이유
Sentry를 먼저 켬	그 뒤에 나는 어떤 오류든 보고되게 하기 위해서입니다
결과를 한 줄로 출력	예약 작업 기록에서 날마다 몇 개를 지웠는지 확인할 수 있습니다

6. 처리방침 버전 올림 (사용자 결정 A, 2026-10-08)

54번의 결정("답변하지 않은 문의는 받은 뒤 1년")을 처리방침에 넣었습니다. 그리고 privacy.html에 적힌 규칙("내용을 바꾸면 버전도 올린다")에 따라 버전을 `2026-10-05`에서 **`2026-10-08`**로 올렸습니다. 아직 공개 전(Draft)이라 다시 동의를 받아야 할 사람은 없고, 앞으로 가입하는 사람은 새 버전으로 기록됩니다.

파일	바꾼 것
`privacy.html` §4	"Messages you send us — 1 year after we reply. **If we never reply, 1 year after you send them.**"와 Version 줄
`home.html` 문의 요약	"One year after we reply — **or one year after you send it, if we never reply** — then we delete it."와 Version 줄
`join.js`, `schemas/auth.py`	`PRIVACY_VERSION = '2026-10-08'`입니다. 가입할 때 화면이 보내고 서버가 확인하는 값이라서, 두 값이 같아야 가입이 됩니다
`docs/Privacy_Policy_Draft.md`	상태, §2.4 보관 표, 버전 줄을 고치고 **버전 기록 표**(10-05 첫 확정판, 10-08 문의 보관)와 PDF를 추가했습니다
`tests/test_auth_service.py`	가입 시험 본문의 `"2026-10-05"`를 상수 `PRIVACY_VERSION`으로 바꿨습니다. 버전이 바뀌어도 시험이 따라가게 하기 위해서입니다

7. 시험 (`tests/test_retention.py`에 2개를 추가해서 모두 13개)

시험	확인한 것
`main()` (세션과 Sentry를 시험용으로 바꿔 끼움)	Sentry를 먼저 켜고, 탈퇴한 지 10일 된 계정과 400일 된 문의를 정리하며, `withdrawn accounts: N, inquiries: M`을 출력합니다
정리 중 오류	예외가 그대로 올라옵니다
(손으로) 실제 명령 실행	`withdrawn accounts: 0, inquiries: 0`이 출력되고, 종료 코드는 0입니다

전체 시험 405개가 통과했고(처리방침 버전을 올린 뒤에도 통과), `ruff` 검사도 통과했습니다.